

PCE Growth Regime Classification and Economic Feature Discovery

An Experimental Supervised Machine Learning Framework for Classifying Personal
Consumption Expenditure (PCE) Growth Using Economic Indicators

Prepared by Yuna Chou

Contribute to IBM Machine Learning Certificate_ Course 3. Model 6_Final Project

Acknowledgment

This project was developed by author with the assistance of ChatGPT, developed by OpenAI, as a supporting tool throughout the analytical and learning process.

ChatGPT was used to support technical discussions, Python code debugging, methodological clarification, idea evaluation, workflow development, interpretation of machine learning concepts and results, and review of the report's structure and written explanations. The collaboration also supported discussions regarding classification threshold selection, class balance, baseline model interpretation, and the design of subsequent machine learning procedures.

The author independently conducted the project, executed and validated the Python code, reviewed the analytical outputs, evaluated the recommendations, and made the final decisions regarding the methodology, interpretations, and conclusions presented in this report.

The use of ChatGPT in this project is therefore acknowledged as AI-assisted analytical and educational support rather than independent authorship or a substitute for the author's judgment and validation.

Overall Project Structure

Problem Definition

Data Preparation

Baseline Difficulty

Supervised Model Learning

Model Comparison

Problem Definition

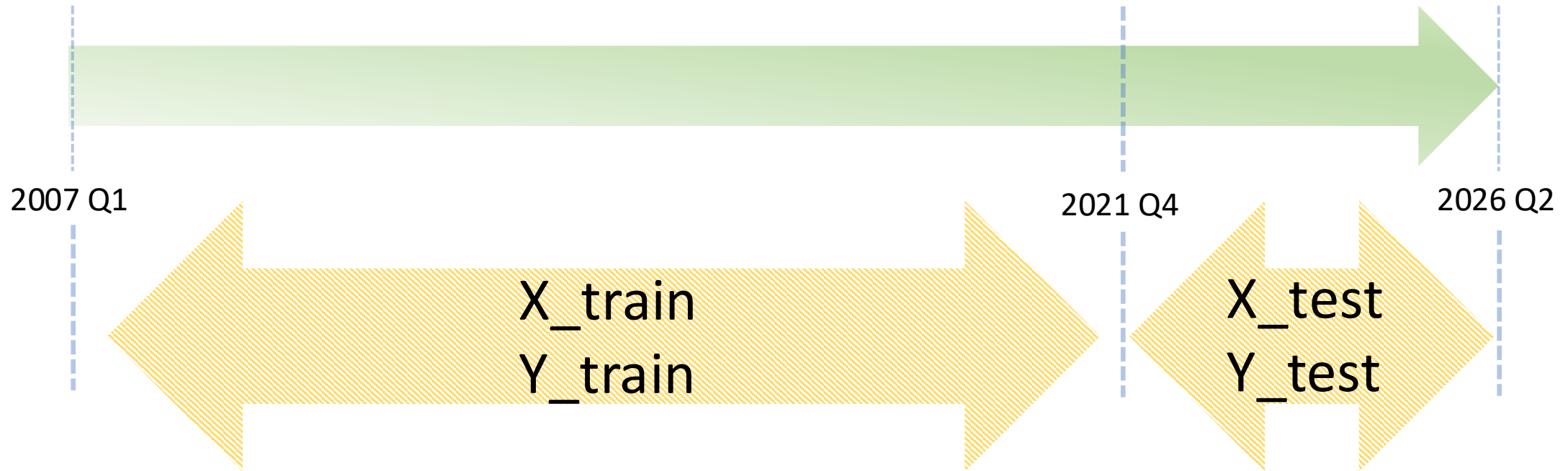
Project Objective

Use historical economic indicators to classify **quarterly Real PCE growth** into **Weak-Growth and Strong-Growth regimes**, systematically compare multiple supervised classification algorithms, identify the best-performing model under chronological out-of-sample testing, and investigate which economic features contain the strongest predictive information.

Data Preparation

Data Description

Chronological train/test split: y (target) & X (predictors)



Variable Description Table

PCE Growth Regime Classification

Table 1. Target and Time Variables

Variable	Role	Economic Category	Description	Expected Relationship / Interpretation
Quarter	Time identifier	Time	Quarter-end date associated with each observation. Used for chronological ordering and the train/test split.	Not used as a predictive feature.
target	Original continuous target	Consumption	Quarterly growth rate of Real Personal Consumption Expenditures (Real PCE).	Higher values represent stronger real consumption growth.
PCE_Regime	Classification target	Consumption	Binary target constructed from Real PCE growth using the training median threshold of 0.5576 .	0 = Weak Growth; 1 = Strong Growth.

Table 2. Economic Predictor Variables

Variable	Economic Category	Description	Expected Relationship / Interpretation
Disposable Personal Income	Income	Personal income remaining after taxes and available for consumption or saving.	Greater household resources may support consumption.
Personal Income	Income	Income received from wages, transfers, investment income, and other sources.	Higher income may be associated with stronger consumption capacity.
Real Disposable Personal Income	Income	Disposable personal income adjusted for inflation.	Measures households' real purchasing power.
Average Hourly Earnings	Labor / Income	Average hourly earnings of workers.	Higher earnings may strengthen household purchasing power.
Weekly Earnings	Labor / Income	Weekly earnings received by workers.	Captures labor-income conditions relevant to consumption.
Wage Growth	Labor / Income	Change in wages over time.	Faster wage growth may support stronger consumption.
Labor Compensation	Labor / Income	Compensation received by workers for labor services.	Represents a broad source of household income.
Self-employment Income	Income	Income generated through self-employment activity.	Captures household income outside conventional employment.
Rental Income	Income / Housing	Income received by property owners from rental activity.	Additional household/property income may affect spending capacity.
Dividend Income	Investment Income	Household income received through dividends.	Captures investment-related household income.
Interest Income	Investment Income	Income households receive from interest-bearing assets.	May affect disposable resources and interact with interest-rate conditions.
Employment Rate	Labor Market	Measure of the proportion of the population employed.	Stronger employment conditions may support consumption.
Full-time Employment	Labor Market	Level or measure of workers employed full-time.	Greater employment stability may support household spending.
Initial Jobless Claims	Labor Market	New claims for unemployment insurance.	Higher claims can signal labor-market deterioration and weaker consumption conditions.
Unemployment Benefits	Transfers / Labor	Benefits paid to eligible unemployed workers.	May partially stabilize household income during labor-market weakness.
Delinquency Rate	Household Credit	Measure of borrowers falling behind on debt payments.	Higher delinquency can indicate household financial stress.
Credit Card & Revolving Loan Growth	Household Credit	Growth in revolving consumer credit such as credit-card balances.	May reflect consumption financing or increasing household financial stress.
Household Assets	Household Wealth	Financial and/or nonfinancial assets held by households.	Greater household wealth may support consumption through wealth effects.
House Price Index	Housing / Wealth	Measure of changes in residential property prices.	Housing appreciation may affect household wealth and confidence.
Vehicle Sales	Consumption / Durable Goods	Measure of automobile and vehicle sales activity.	Provides information about durable-goods demand and household spending conditions.
Consumer Sentiment Index	Consumer Expectations	Measure of household attitudes toward economic and financial conditions.	Higher sentiment may be associated with greater willingness to spend.
Federal Funds Rate	Monetary Policy	Key short-term policy interest rate influenced by the Federal Reserve.	Affects borrowing costs, saving incentives, financial conditions, and economic activity.
Corporate Bond Yield	Financial Conditions	Yield required on corporate debt.	Higher yields may indicate tighter financing conditions and/or greater perceived risk.
Exchange Rate	International / Financial	Measure of the value of the U.S. dollar relative to other currencies in the dataset.	May relate to import prices, inflation, purchasing power, and broader macroeconomic conditions.
Market Volatility (VIX)	Financial Markets	Measure of expected stock-market volatility.	Higher volatility can represent greater financial uncertainty and potentially more cautious spending.
Population	Demographics	Population level representing changes in the potential consumer base.	Population changes can influence aggregate consumption and long-run economic trends.

Baseline Difficulty

Step 1. PCE Target Distribution & Classification Target Design

➤ 1.1 Download Real Personal Consumption Expenditures from FRED

```
PCE_URL = (  
    "https://fred.stlouisfed.org/graph/fredgraph.csv"  
    "?id=PCEC96"  
)  
  
pce = pd.read_csv(PCE_URL)  
  
print("Raw FRED data:")  
display(pce.head())
```

Raw FRED data:

	observation_date	PCEC96
0	2007-01-01	11181.0
1	2007-02-01	11178.2
2	2007-03-01	11190.7
3	2007-04-01	11201.5
4	2007-05-01	11218.0

Changed
Column names

```
pce.columns = ["DATE", "Real_PCE"]  
  
pce["DATE"] = pd.to_datetime(pce["DATE"])  
pce["Real_PCE"] = pd.to_numeric(pce["Real_PCE"], errors="coerce")  
  
pce = pce.dropna(subset=["Real_PCE"])  
  
print("\nCleaned data:")  
display(pce.head())
```

Cleaned data:

	DATE	Real_PCE
0	2007-01-01	11181.0
1	2007-02-01	11178.2
2	2007-03-01	11190.7
3	2007-04-01	11201.5
4	2007-05-01	11218.0

Step 1. PCE Target Distribution & Classification Target Design

➤ 1.2 2007Q1 – 2026Q2 (Redefine frequency from monthly to quarterly)

```
pce = pce.set_index("DATE").sort_index()

print("\nFrequency / observations:")
print(pce.index.min(), "to", pce.index.max())
print("Number of observations:", len(pce))
```

Frequency / observations:
2007-01-01 00:00:00 to 2026-06-01 00:00:00
Number of observations: 234

	DATE	Real_PCE
0	2007-01-01	11181.0
1	2007-02-01	11178.2
2	2007-03-01	11190.7
3	2007-04-01	11201.5
4	2007-05-01	11218.0

Covered:

1. Date: Monthly -> Quarterly
2. Added a new column to present Real PCE growth rate quarterly

Target dataset shape: (77, 2)

First observations:

DATE	Real_PCE	Real_PCE_Growth_QoQ
2007-06-30	11218.5	0.248421
2007-09-30	11309.6	0.812052
2007-12-31	11335.1	0.225472

Step 1. PCE Target Distribution & Classification Target Design

➤ 1.3 Real_PCE_Growth QoQ Descriptive Statistics

```
[10]: target_stats = df[TARGET].describe()

[11]: print("PCE Growth Descriptive Statistics")
      print("=" * 40)
      print(target_stats)

      print("\nAdditional statistics")
      print("=" * 40)
      print(f"Median:      {df[TARGET].median():.4f}")
      print(f"Std Dev:    {df[TARGET].std():.4f}")
      print(f"Minimum:   {df[TARGET].min():.4f}")
      print(f"Maximum:   {df[TARGET].max():.4f}")
      print(f"Skewness:  {df[TARGET].skew():.4f}")
```

```
PCE Growth Descriptive Statistics
=====
count      77.000000
mean        0.541667
std         1.095797
min         -6.470559
25%         0.286710
50%         0.564123
75%         0.826364
max         4.657546
Name: Real_PCE_Growth_QoQ, dtype: float64

Additional statistics
=====
Median:     0.5641
Std Dev:    1.0958
Minimum:    -6.4706
Maximum:    4.6575
Skewness:   -2.5745
```

Step 1. PCE Target Distribution & Classification Target Design

➤ 1.4 Examine positive and negative PCE growth:

PCE Growth ≤ 0 vs PCE Growth > 0

```
[13]: negative = (df[TARGET] < 0).sum()
      zero = (df[TARGET] == 0).sum()
      positive = (df[TARGET] > 0).sum()

      total = df[TARGET].notna().sum()

      growth_summary = pd.DataFrame({
          "Group": [
              "Negative PCE Growth",
              "Zero PCE Growth",
              "Positive PCE Growth"
          ],
          "Count": [
              negative,
              zero,
              positive
          ]
      })

      growth_summary["Percentage"] = (
          growth_summary["Count"] / total * 100
      ).round(2)

      print(growth_summary)
```

Output:

	Group	Count	Percentage
0	Negative PCE Growth	10	12.99
1	Zero PCE Growth	0	0.00
2	Positive PCE Growth	67	87.01

Comments:

The result is imbalanced for the primary classification target.

- Negative: 10
- Positive: 67

Step 1. PCE Target Distribution & Classification Target Design

➤ 1.5 Examine candidate statistical thresholds

```
[14]: candidate_thresholds = pd.Series({
        "Zero": 0,
        "25th Percentile": df[TARGET].quantile(0.25),
        "Median": df[TARGET].median(),
        "75th Percentile": df[TARGET].quantile(0.75),
        "Mean": df[TARGET].mean()
    })

print("Candidate PCE Classification Thresholds")
print("=" * 45)
print(candidate_thresholds)
```

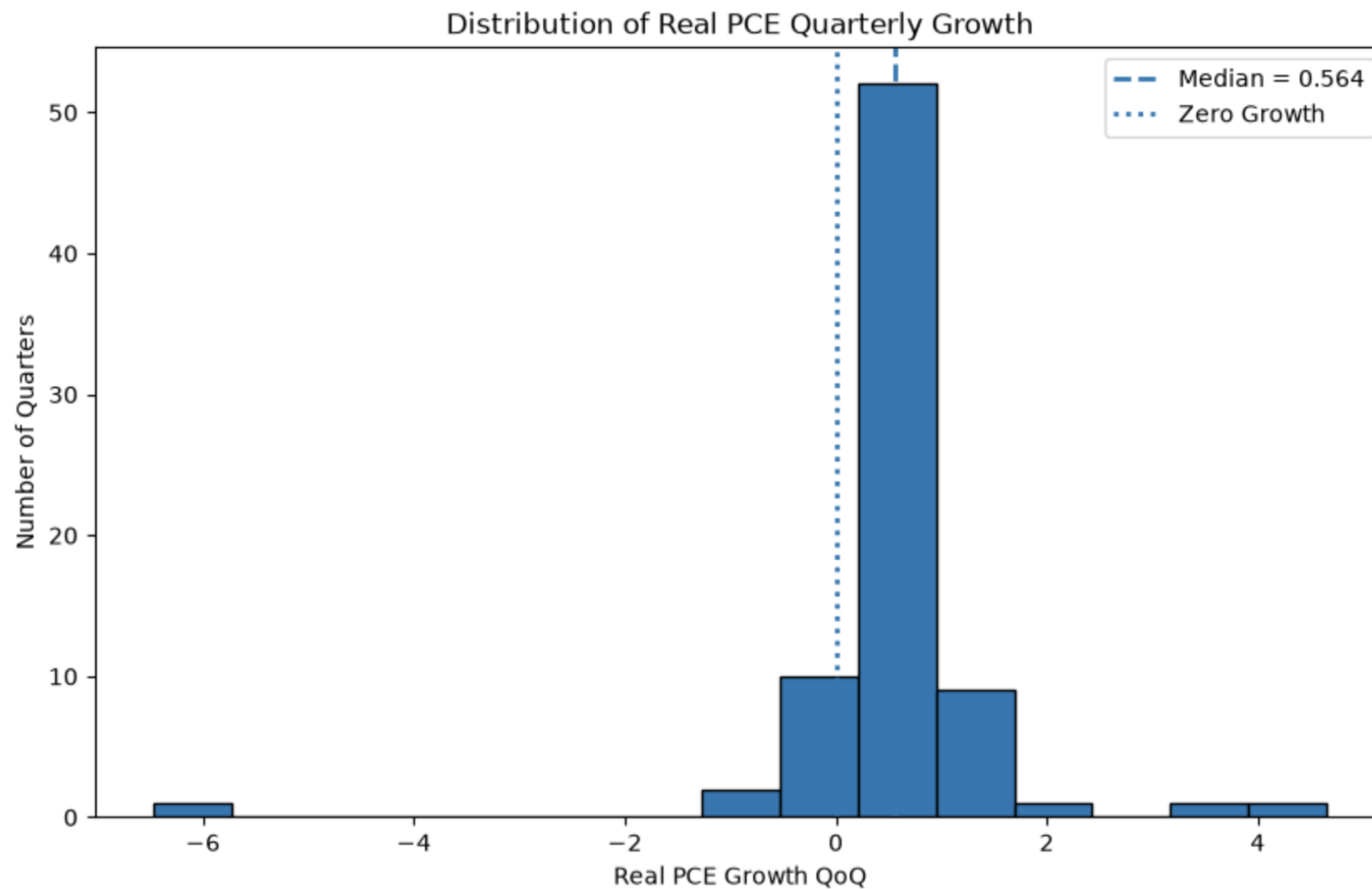
Candidate PCE Classification Thresholds	
=====	
Zero	0.000000
25th Percentile	0.286710
Median	0.564123
75th Percentile	0.826364
Mean	0.541667
dtype: float64	

➤ 1.6 The Resulting Class Balance

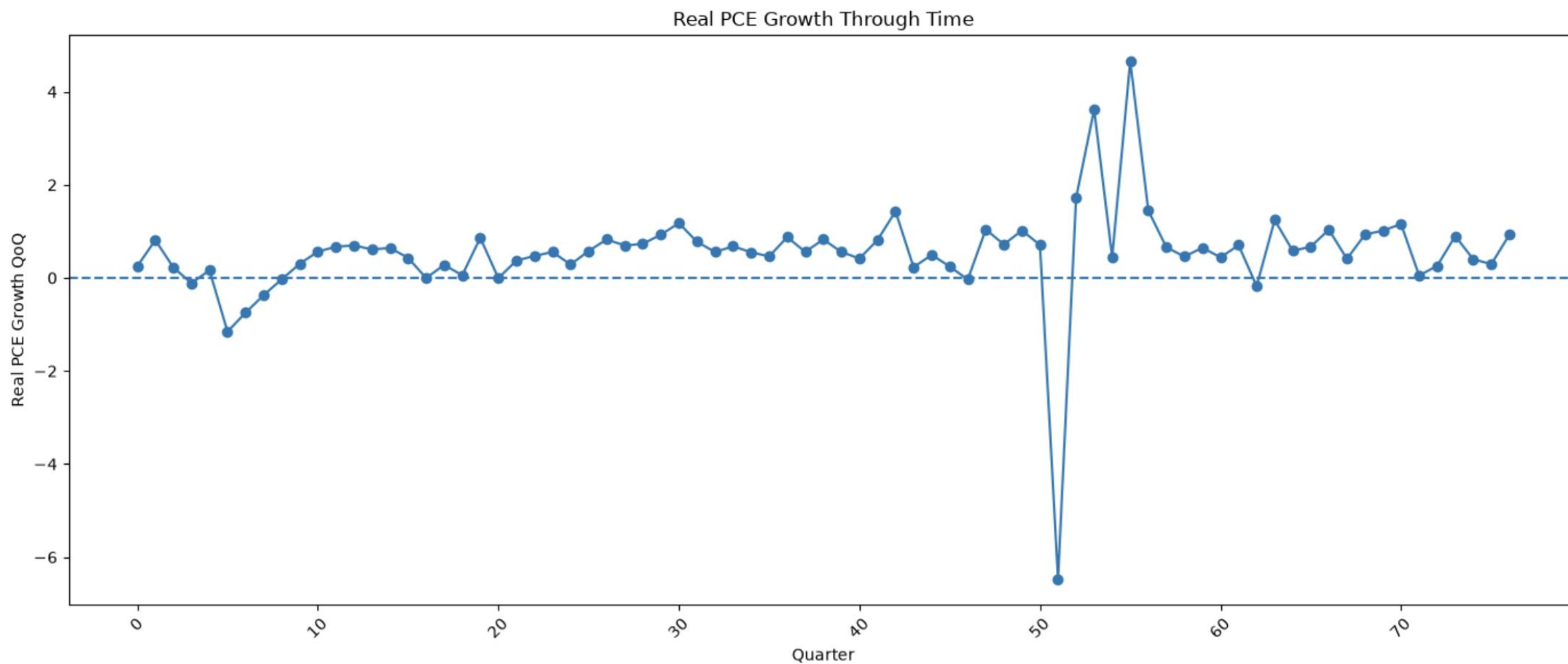
	Threshold_Method	Threshold	Class_0_Count	Class_1_Count	Class_0_Percent	Class_1_Percent
0	Zero	0.0000	10	67	12.99	87.01
1	25th Percentile	0.2867	20	57	25.97	74.03
2	Median	0.5641	39	38	50.65	49.35
3	75th Percentile	0.8264	58	19	75.32	24.68
4	Mean	0.5417	33	44	42.86	57.14

Threshold	Weak PCE (Class_0)	Strong PCE (Class_1)	
Zero	Too Few (12.99%) Only 10 quarters → extremely imbalanced.	Too Many (87.01%) 67 quarters → extremely imbalanced.	
25th percentile	Low (25.97%) 20 quarters → still imbalanced toward strong class.	High (74.03%) 57 quarters → still imbalanced.	
Median	Balanced (50.65%) 39 quarters → very close to 50/50.	Balanced (49.35%) 38 quarters → very close to 50/50.	
75th percentile	Too Many (75.32%) 58 quarters → weak class dominates.	Too Few (24.68%) 19 quarters → too few strong quarters.	
Mean	Balanced (42.86%) 33 quarters → reasonably balanced.	Balanced (57.14%) 44 quarters → reasonably balanced.	

➤ 1.7 Check the Distribution of this classification threshold



➤ 1.8 Inspect PCE through time



The Conclusion of Step 1 and The Purpose of Next Step

The Distinguish the Exploratory Threshold from the Official Threshold

The Training Median Selected Structure:



Step 2. Create the Official Classification Target

➤ 2.1 create the target using only the **training-derived median**.

- The classification threshold is calculated from the training period only to prevent information from the future test period from influencing target construction.

```
[22]: # Threshold derived ONLY from training data
pce_threshold = train_df[TARGET].median()

print(f"PCE classification threshold: {pce_threshold:.6f}%")
```

PCE classification threshold: 0.557566%

```
[24]: train_df["PCE_Regime"].value_counts()
```

```
[24]: PCE_Regime
0      30
1      29
Name: count, dtype: int64
```

➤ 2.2 Check the class distribution in training dataset and testing dataset

```
class_summary(train_df, "TRAINING CLASS DISTRIBUTION")  
class_summary(test_df, "TESTING CLASS DISTRIBUTION")
```

TRAINING CLASS DISTRIBUTION

```
=====
```

	Count	Percentage
Class 0 – Weak PCE Growth	30	50.85
Class 1 – Strong PCE Growth	29	49.15

TESTING CLASS DISTRIBUTION

```
=====
```

	Count	Percentage
Class 0 – Weak PCE Growth	7	38.89
Class 1 – Strong PCE Growth	11	61.11

➤ 2.3 Threshold = Median (Y_{train}) = 0.5576

- Class 0: Weak PCE Growth
- Class 1: Strong PCE Growth

```
26]: comparison = pd.DataFrame({
    "PCE_Growth": test_df[TARGET],
    "PCE_Regime": test_df["PCE_Regime"]
})

comparison["Regime_Label"] = comparison["PCE_Regime"].map({
    0: "Weak Growth",
    1: "Strong Growth"
})

print(comparison)
```

DATE	PCE_Growth	PCE_Regime	Regime_Label
2022-03-31	0.648109	1	Strong Growth
2022-06-30	0.440726	0	Weak Growth
2022-09-30	0.702199	1	Strong Growth
2022-12-31	-0.175467	0	Weak Growth
2023-03-31	1.244152	1	Strong Growth
2023-06-30	0.581515	1	Strong Growth
2023-09-30	0.671197	1	Strong Growth
2023-12-31	1.036415	1	Strong Growth
2024-03-31	0.410061	0	Weak Growth
2024-06-30	0.930493	1	Strong Growth
2024-09-30	1.017156	1	Strong Growth
2024-12-31	1.156657	1	Strong Growth
2025-03-31	0.051171	0	Weak Growth
2025-06-30	0.258160	0	Weak Growth
2025-09-30	0.898805	1	Strong Growth
2025-12-31	0.399656	0	Weak Growth
2026-03-31	0.296151	0	Weak Growth
2026-06-30	0.927072	1	Strong Growth

Step 3. Class Balance & Baseline Difficulty Analysis

The purpose of Step 3 is to verify that the **classification target is reasonably balanced** and establish a minimum performance benchmark before training machine-learning models. With **30 Weak Growth** and **29 Strong Growth** observations, the classes are nearly balanced, and **the majority-class baseline accuracy is approximately 50.8%.**

Therefore, future models should meaningfully outperform this baseline to demonstrate that the economic features contain useful predictive information for distinguishing between Weak and Strong PCE growth regimes.

➤ **3.1** Define an Imbalanced Ratio (IR)

$$IR = \frac{\text{majority class count}}{\text{minority class count}}$$

Interpretation: **Imbalance Ratio Interpretation**

1.00–1.25 Very balanced

1.25–1.50 Mild imbalance

1.50–2.00 Moderate

2.00+ Significant

4.00+ Severe

➤ 3.1 Define an Imbalanced Ratio (IR)

$$IR = \frac{\text{majority class count}}{\text{minority class count}}$$

Interpretation:

Imbalance Ratio	Interpretation
1.00–1.25	Very balanced
1.25–1.50	Mild imbalance
1.50–2.00	Moderate
2.00+	Significant
4.00+	Severe

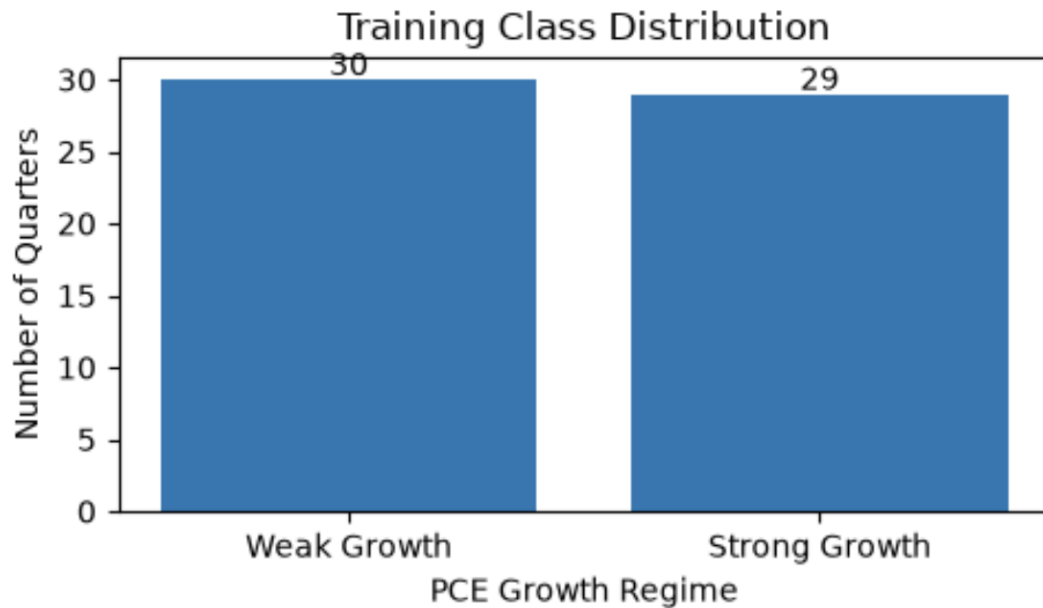
Result

```
def imbalance_ratio(y):  
    counts = y.value_counts()  
  
    majority = counts.max()  
    minority = counts.min()  
  
    return majority / minority  
  
train_ir = imbalance_ratio(y_train)  
test_ir = imbalance_ratio(y_test)  
  
print(f"Training imbalance ratio: {train_ir:.3f}")  
print(f"Testing imbalance ratio : {test_ir:.3f}")  
Training imbalance ratio: 1.034  
Testing imbalance ratio : 1.571
```

➤ 3.2 Visualize the distributions

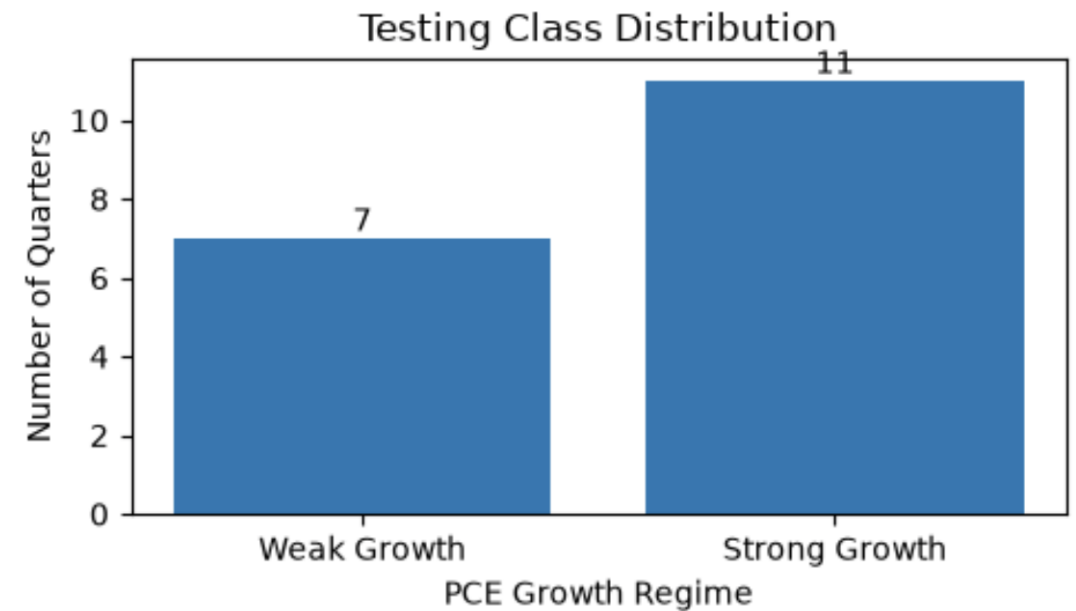
Training data

```
plt.tight_layout()  
plt.show()
```



Testing data

```
plt.tight_layout()  
plt.show()
```



➤ 3.3 Establish a naïve baseline

Dummy Classifier Baseline

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(
    strategy="most_frequent"
)

# Dummy X values are sufficient here because
# the classifier intentionally ignores predictors.
X_dummy_train = np.zeros((len(y_train), 1))
X_dummy_test  = np.zeros((len(y_test), 1))

dummy.fit(
    X_dummy_train,
    y_train
)

dummy_pred = dummy.predict(
    X_dummy_test
)

print("Dummy prediction:")
print(dummy_pred)
```

```
Dummy prediction:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Dummy Classifier Baseline Report

DUMMY CLASSIFIER BASELINE

```
=====
Accuracy:          0.3889
Precision:         0.0000
Recall:            0.0000
F1 Score:          0.0000
Balanced Accuracy: 0.5000
```

Confusion Matrix:

```
[[ 7  0]
 [11  0]]
```

Classification Report:

	precision	recall	f1-score	support
Weak Growth	0.39	1.00	0.56	7
Strong Growth	0.00	0.00	0.00	11
accuracy			0.39	18
macro avg	0.19	0.50	0.28	18
weighted avg	0.15	0.39	0.22	18

➤ 3.4 Add a second baseline- STRATIFIED RANDOM BASELINE

```
dummy_stratified = DummyClassifier(  
    strategy="stratified",  
    random_state=42)  
  
dummy_stratified.fit(  
    X_dummy_train,  
    y_train)  
stratified_pred = dummy_stratified.predict(  
    X_dummy_test)  
  
print("STRATIFIED DUMMY CLASSIFIER")  
print("=" * 45)  
print(  
    f"Accuracy: "  
    f"{accuracy_score(y_test, stratified_pred):.4f}"  
)  
print(  
    f"F1 Score: "  
    f"{f1_score(y_test, stratified_pred, zero_division=0):.4f}"  
)  
print(  
    f"Balanced Accuracy: "  
    f"{balanced_accuracy_score(y_test, stratified_pred):.4f}"  
)  
print("\nConfusion Matrix:")  
print(  
    confusion_matrix(  
        y_test,  
        stratified_pred))
```

Report

STRATIFIED DUMMY CLASSIFIER

Accuracy: 0.4444

F1 Score: 0.5000

Balanced Accuracy: 0.4416

Confusion Matrix:

[[3 4]

[6 5]]

Current Findings

The initial analysis demonstrates that defining PCE regimes simply as negative versus positive growth produces a substantially imbalanced classification target. Alternative classification thresholds were therefore evaluated to identify a more appropriate boundary between weak and strong PCE growth.

To prevent information leakage, the official binary classification threshold was derived exclusively from the training period. A **training median threshold of 0.5576% quarterly Real PCE growth was selected**, defining observations below or equal to the threshold as **Weak Growth (Class 0)** and observations above the threshold as **Strong Growth (Class 1)**.

The resulting training classes contain **30 Weak Growth observations** and **29 Strong Growth observations**, indicating an approximately balanced classification problem. Consequently, **resampling is not currently required**.

The majority-class baseline is approximately 50.8%. Future machine learning models should meaningfully outperform this baseline to provide evidence that the selected economic features contain useful predictive information for distinguishing weak from strong PCE growth regimes.

These findings establish the classification target and evaluation benchmark but **do not yet demonstrate which economic features are predictive or whether the final classification model will generalize successfully to future observations**.

PCE Regime for y Target Definition

$$PCE_Regime = \begin{cases} 0, & target \leq 0.5576 \\ 1, & target > 0.5576 \end{cases}$$

Supervised Model Learning

Models Selected

Model	Learning idea
Dummy	No useful X information
Logistic Regression	Linear probability boundary
KNN	Similar economic conditions
Linear SVM	Maximum-margin boundary
Decision Tree	Nonlinear threshold rules
Random Forest	Bagged nonlinear trees
XGBoost	Boosted nonlinear trees

Logistic Regression

Metric	Logistic Regression	Interpretation
Accuracy	0.5556	55.56% of test quarters classified correctly
Balanced Accuracy	0.575	Average performance across Weak and Strong classes is 57.5%
Precision	0.6667	When predicting Strong Growth, 66.67% were actually Strong
Recall	0.4000	It identified only 40% of the actual Strong-Growth quarters
F1	0.5000	Moderate/weak balance between precision and recall
ROC-AUC	0.5500	Only slightly better than random ranking

Interpretation:

Logistic Regression modestly outperformed the naïve DummyClassifier baselines in accuracy and balanced accuracy, suggesting that the economic predictors contain some linear classification signal for distinguishing Weak and Strong PCE growth regimes. However, the model achieved only 40% recall for the Strong-Growth class and an ROC-AUC of 0.55, indicating weak overall discrimination. Therefore, Logistic Regression provides a useful interpretable benchmark, but its predictive performance is not sufficiently strong to serve as the final model.

KNN

Metric	KNN	Interpretation
Accuracy	0.5556	55.56% of test quarters were classified correctly
Balanced Accuracy	0.5000	Average performance across Weak and Strong classes was 50.0%
Precision	0.5556	When predicting Strong Growth, 55.56% were actually Strong
Recall	1.0000	It identified 100% of the actual Strong-Growth quarters
F1	0.7143	High F1 resulted from perfect Strong-Growth recall
ROC-AUC	0.5000	Probability ranking was equivalent to random discrimination

Interpretation:

KNN predicted every test quarter as Strong Growth. Therefore, its high recall and F1 should not be interpreted as strong overall classification performance.

Linear SVM

Metric	Linear SVM	Interpretation
Accuracy	0.4444	44.44% of test quarters were classified correctly
Balanced Accuracy	0.4250	Average performance across Weak and Strong classes was 42.5%
Precision	0.5000	When predicting Strong Growth, 50% were actually Strong
Recall	0.6000	It identified 60% of the actual Strong-Growth quarters
F1	0.5455	Moderate balance between precision and recall
ROC-AUC	0.5750	Probability ranking was slightly better than random

Interpretation:

Linear SVM did not collapse completely to one class, but its balanced accuracy below 0.50 indicates **weak overall class discrimination**.

Decision Tree

Decision Tree V1

Metric	Decision Tree V1	Interpretation
Accuracy	0.5556	55.56% of test quarters were classified correctly
Balanced Accuracy	0.5000	Average performance across Weak and Strong classes was 50.0%
Precision	0.5556	When predicting Strong Growth, 55.56% were actually Strong
Recall	1.0000	It identified 100% of the actual Strong-Growth quarters
F1	0.7143	High F1 resulted from perfect Strong-Growth recall
ROC-AUC	0.4875	Probability ranking was slightly worse than random

interpretation:

Like KNN, Decision Tree V1 predicted all 18 observations as Strong Growth. Therefore, its apparently high F1 score does not represent balanced classification ability.

Decision Tree V2

Metric	Decision Tree V2	Interpretation
Accuracy	0.4444	44.44% of test quarters were classified correctly
Balanced Accuracy	0.4875	Average performance across Weak and Strong classes was 48.75%
Precision	0.5000	When predicting Strong Growth, 50% were actually Strong
Recall	0.1000	It identified only 10% of the actual Strong-Growth quarters
F1	0.1667	Very weak balance between precision and recall
ROC-AUC	0.4875	Probability ranking was slightly worse than random

Interpretation:

V2 reduced the strong-class prediction tendency of V1, but at the cost of missing most actual Strong-Growth quarters.

Random Forest

Metric	Random Forest	Interpretation
Accuracy	0.5000	50% of test quarters were classified correctly
Balanced Accuracy	0.4750	Average performance across Weak and Strong classes was 47.5%
Precision	0.5385	When predicting Strong Growth, 53.85% were actually Strong
Recall	0.7000	It identified 70% of the actual Strong-Growth quarters
F1	0.6087	Moderate balance between precision and recall
ROC-AUC	0.4000	Probability ranking was worse than random discrimination

Interpretation:

Random Forest captured more Strong-Growth observations than Logistic Regression but produced poorer overall class discrimination and the lowest ROC-AUC among these models.

XGBoost

Metric	XGBoost	Interpretation
Accuracy	0.5556	55.56% of test quarters were classified correctly
Balanced Accuracy	0.5750	Average performance across Weak and Strong classes was 57.5%
Precision	0.6667	When predicting Strong Growth, 66.67% were actually Strong
Recall	0.4000	It identified 40% of the actual Strong-Growth quarters
F1	0.5000	Moderate/weak balance between precision and recall
ROC-AUC	0.6000	Best probability-ranking performance among the tested models

Interpretation:

XGBoost tied Logistic Regression on the classification metrics but achieved a higher ROC-AUC (**0.60 vs. 0.55**), suggesting somewhat better ranking of Weak versus Strong Growth probabilities.

Model Comparison

Overall Comparison

Model (Strong Growth)	Accuracy	Balanced Accuracy	Precision	Recall	F1	ROC-AUC
Most-Frequent Dummy	0.389	0.500	0.000	0.000	0.000	—
Stratified Dummy	0.444	0.442	—	—	0.500	—
Logistic Regression	0.5556	0.575	0.6667	0.4	0.500	0.55
KNN	0.5556	0.500	0.5556	1.000	0.7143	0.500
SVM (Linear)	0.4444	0.425	0.5000	0.6	0.5455	0.575
Decision Tree V1	0.5556	0.500	0.5556	1.0	0.7143	0.4875
Decision Tree V2	0.4444	0.4875	0.5	0.1	0.1667	0.4875
Random Forest	0.500	0.4750	0.5385	0.7	0.6087	0.4
XGBoost	0.5556	0.5750	0.6667	0.4	0.500	0.6

Conclusion

There is no model demonstrates strong out-of-sample classification performance yet. Logistic Regression and XGBoost have the highest Balanced Accuracy (0.575), while XGBoost has the highest ROC-AUC (0.600). KNN and Decision Tree V1 appear strong on Recall/F1 but actually collapsed toward predicting Strong Growth, showing why we need to evaluate multiple metrics and confusion matrices together, rather than selecting a model from Accuracy or F1 alone

Special Insights

Decision Tree V2_ the result of walk-forward stability analysis

	Fold	Train_Start	Train_End	Validation_Start	Validation_End	Balanced_Accuracy	Predicted_Weak	Predicted_Strong
0	1	2007-06-30	2010-09-30	2010-12-31	2012-12-31	0.500000	9	0
1	2	2007-06-30	2012-12-31	2013-03-31	2015-03-31	0.500000	9	0
2	3	2007-06-30	2015-03-31	2015-06-30	2017-06-30	0.500000	0	9
3	4	2007-06-30	2017-06-30	2017-09-30	2019-09-30	0.500000	9	0
4	5	2007-06-30	2019-09-30	2019-12-31	2021-12-31	0.928571	3	6

Decision Tree V2_ I matched the timeline alongside economic history

Approx. period	Regime label	Monetary policy	Fiscal policy	PCE relevance
2007–2009	Financial Crisis / Recession	Rapid easing → zero rates + QE	Strong emergency stimulus	Consumption contraction, credit stress, unemployment shock
2010–2014	Post-Crisis Recovery	Zero rates + QE	Stimulus fading → fiscal restraint	Slow household recovery, deleveraging
2015–2017	Normalization / Expansion	Gradual rate increases	Relatively moderate → tax-policy transition	Stronger labor market and consumption
2018–2019	Late Expansion	Tightening → 2019 easing	TCJA / expansionary fiscal influence	Strong consumption but slowing macro conditions
2020–2021	COVID Shock / Stimulus Recovery	Zero rates + massive QE	Extraordinary fiscal stimulus	Extremely abnormal PCE behavior

Interpretation of Special Insights

Decision Tree V2 used walk-forward stability analysis instead of extensive hyperparameter tuning to test whether the model remained reliable across different economic periods.

The first four folds achieved 0.50 balanced accuracy and often predicted only one class, including unstable classification during normal economic conditions. In contrast, Fold 5 achieved 0.929 balanced accuracy during the COVID shock and recovery.

This suggests that **Decision Tree V2 is regime-dependent**: it struggles with subtle PCE growth differences during normal periods but detects large economic shifts more effectively. Walk-forward analysis therefore helps identify temporal stability and genuine economic signals, rather than simply optimizing model parameters.

Reference

Data Sources

U.S. Bureau of Economic Analysis. (2026). *Real Personal Consumption Expenditures [PCEC96]*. Federal Reserve Bank of St. Louis, FRED. Retrieved August 2026.

Federal Reserve Bank of St. Louis. (2026). *FRED Economic Data*. Real Personal Consumption Expenditures and related economic time-series data.

Software and Technical Documentation

pandas development team. (2026). *pandas documentation*. pandas/NumFOCUS.

NumPy Developers. (2026). *NumPy documentation*.

Matplotlib Development Team. (2026). *Matplotlib documentation*.

scikit-learn developers. (2026). *scikit-learn: Machine Learning in Python — User Guide and API Documentation*. Functions and methods referenced in this project include classification metrics, confusion matrices, baseline classifiers, and machine learning model evaluation.

AI-Assisted Research and Development

OpenAI. (2026). *ChatGPT*. AI-assisted support used for Python debugging, methodological discussion, workflow development, idea evaluation, interpretation of machine learning results, and review of project documentation.